

FDCAN implementation

Here I define the design of the CAN primitives used in the bootloader.

Overview of the protocol

Header

Definition

There is no header in the TCP/IP sense, but there are arbitration & control fields, and the driver/HAL sometimes add metadata

Arbitration field

The arbitration field holds:

- An identifier (11 or 29 bits) → **used for addressing, routing and metadata**
- **RTR** (Remote Transmission Request), always **0** in FDCAN

Control field

The control field holds the following flags:

- **IDE**: standard vs extended (11 or 29 bits?)
- **EDL**: FD Frame Indicator
- **BRS**: Bit Rate Switch
- **ESI**: Error State Indicator
- **DLC**: Data Length Code (0-15)

DLC and payload length

For the first **8** bytes, DLC equals the payload length. Then, **DLC** is incremented by 1 each 4 bytes added

Payload

Definition

The payload is up to **64 bytes** big, and carries meaning

CAN Arbitration Field Specification (11-bit Standard ID)

Semantics?

The meaning of the bits inside the arbitration field are entirely up to the developer. Thus, it is crucial to define strict semantics for the field beforehand, and apply them consistently

Priority

The only rule enforced by default is that messages with a lower arbitration field always have priority over the others

- **ID width**: 11 bits (standard CAN)
- **Purpose**: Encodes bus-level arbitration, message priority, message class, and target node.

Field Layout (MSB → LSB)

Bits	Field	Width (bits)	Description
10-8	Priority	3	Bus arbitration priority (lower ID = higher priority)
7-4	Message Class	4	Encodes message type; e.g., bootloader, runtime, diagnostic
3-0	Node ID	4	Target node address; 0-15 nodes supported, reserved IDs may be used for broadcast

Notes

- **Priority:** Determines arbitration on the bus; smaller numeric values have higher bus priority.
- **Message Class:** Logical classification of the message; detailed semantics defined [here](#).
- **Node ID:** Identifies destination node; a reserved value can be used to indicate broadcast messages.
- **Total IDs:** Up to 2048 possible IDs (11-bit), ample for the current system topology.
- **Scope:** Single master (Jetson) to multiple MCUs; individual MCUs can be flashed separately.
- **Transport:** Compatible with CAN-FD; ID is independent of payload size or FDCAN-specific flags.

🔍 Special values

- There is the `NO_NODE_ID` special value for the Node ID field (`0xF`) meant to be used when an MCU does not have a node id assigned yet
- There is the `BROADCAST_ID` special value for the Node ID field (`0xE`) meant to be used when broadcasting a message to all MCUs

Primitives

🔍 Info

The following *high-level* primitives are implemented using the FDCAN HAL provided by STM32

⚠️ Responsibilities

The following function must be *dumb primitives*, and only implement what their name states. It would be bad design to assign them application- or boot-level responsibilities

CAN_send

🔗 Definition

The `CAN_send` function has the following signature:

```
CAN_TX_STATE CAN_send(CAN_message *msg);
```

It shall send the given `msg` to master. **It is a non-blocking function.**

It returns a `CAN_TX_STATE` variable, where:

- `OK` means the message was successfully queued for transmission
- Anything else is an error (tbd)

More specifically, this function queues the given message in the `FDCAN TX FIFO` and returns `OK` if this operation was successful. The `FDCAN` controller will take it from there, and will send the message whenever possible. It handles all bus arbitration and retries and correctness, so there is no need to re-send the message *ever*.

🔍 Possible errors?

Probably, the only error needed at application level is `FIFO_FULL`. In that case, we shall retry later *at application level*. We may have `BUS_OFF` or `NOT_INITIALIZED` errors, but let's add them later

CAN_receive

Definition

The `CAN_receive` function has the following signature:

```
CAN_RX_STATE CAN_receive(CAN_message *rsp);
```

It shall receive a message from the bus and copy it into `rsp`.

It is a non-blocking function.

It returns a `CAN_RX_STATE` variable, where:

- `OK` means the message was successfully received and can be correctly read inside `rsp`
- Anything else is an error (tbd)

More specifically, this primitive reads from `FDCAN_RX_FIFO` buffer, and:

- If the `FIFO` is empty, returns `NO_MESSAGE` and `rsp` is not touched.
- If the `FIFO` is not empty, copy them into `rsp`, return `OK`
- If any error arises, return. `rsp` is not valid in this case and should not be read

Invariant

This function always leaves `rsp` in a valid state, as long as it returns `OK`

Possible errors?

I guess `NO_MESSAGE` is a return state on its own not an error, and `UNKNOWN_ERROR` may cover the rest for now

Bootloader protocol

Table of `boot_msg_class_t` types

Definition

A `boot_msg_class_t` is a 4 bits code stating the purpose of the message

Every CAN message has an ID. Here are the custom defined ones and their meaning:

- `BOOT_FW_REQUEST`, `MCU → Jetson`. Request to receive new firmware. The payload should be the UID of the MCU (96 bits).
- `BOOT_FW_ASSIGN`, `Jetson → MCU`. Assign node-ID and signal intentions to start sending firmware. First 96 bits are the UID of the intended receiver, last 8 bits are node ID.
- `BOOT_FW_DATA`, `Jetson → MCU`.
- `BOOT_FW_END`, `Jetson → MCU`.
- `BOOT_FW_ERROR`, `Jetson ↔ MCU`.